JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.invoke

Class MethodHandleProxies

java.lang.Object
    java.lang.invoke.MethodHandleProxies

```
public class MethodHandleProxies
extends Object
```

This class consists exclusively of static methods that help adapt method handles to other JVM types, such as interfaces.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
static <T> T	**asInterfaceInstance**(**Class**<T> intfc, **MethodHandle** target)	Produces an instance of the given single-method interface which redirects its calls to the given method handle.
static boolean	**isWrapperInstance**(**Object** x)	Determines if the given object was produced by a call to **asInterfaceInstance**.
static **MethodHandle**	**wrapperInstanceTarget**(**Object** x)	Produces or recovers a target method handle which is behaviorally equivalent to the

		unique method of this wrapper instance.
static Class<?>	wrapperInstanceType(Object x)	Recovers the unique single-method interface type for which this wrapper instance was created.

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Method Detail

asInterfaceInstance

```
public static <T> T asInterfaceInstance(Class<T> intfc,
                                        MethodHandle target)
```

Produces an instance of the given single-method interface which redirects its calls to the given method handle.

A single-method interface is an interface which declares a uniquely named method. When determining the uniquely named method of a single-method interface, the public `Object` methods (`toString`, `equals`, `hashCode`) are disregarded. For example, `Comparator` is a single-method interface, even though it re-declares the `Object.equals` method.

The interface must be public. No additional access checks are performed.

The resulting instance of the required type will respond to invocation of the type's uniquely named method by calling the given target on the incoming arguments, and returning or throwing whatever the target returns or throws. The invocation will be as if by `target.invoke`. The target's type will be checked before the instance is created, as if by a call to `asType`, which may result in a `WrongMethodTypeException`.

The uniquely named method is allowed to be multiply declared, with distinct type descriptors. (E.g., it can be overloaded, or can possess bridge methods.) All such declarations are connected directly to the target method handle. Argument and return types are adjusted by `asType` for each individual declaration.

The wrapper instance will implement the requested interface and its super-types, but no other single-method interfaces. This means that the instance will not unexpectedly pass an `instanceof` test for any unrequested type.

Implementation Note: Therefore, each instance must implement a unique single-method interface. Implementations may not bundle together multiple single-method interfaces onto single implementation classes in the style of `AWTEventMulticaster`.

The method handle may throw an *undeclared exception*, which means any checked exception (or other checked throwable) not declared by the requested type's single abstract method. If this happens, the throwable will be wrapped in an instance of `UndeclaredThrowableException` and thrown in that wrapped form.

Like `Integer.valueOf`, `asInterfaceInstance` is a factory method whose results are defined by their behavior. It is not guaranteed to return a new instance for every call.

Because of the possibility of bridge methods and other corner cases, the interface may also have several abstract methods with the same name but having distinct descriptors (types of returns and parameters). In this case, all the methods are bound in common to the one given target. The type check and effective `asType` conversion is applied to each method type descriptor, and all abstract methods are bound to the target in common. Beyond this type check, no further checks are made to determine that the abstract methods are related in any way.

Future versions of this API may accept additional types, such as abstract classes with single abstract methods. Future versions of this API may also equip wrapper instances with one or more additional public "marker" interfaces.

If a security manager is installed, this method is caller sensitive. During any invocation of the target method handle via the returned wrapper, the original creator of the wrapper (the caller) will be visible to context checks requested by the security manager.

Type Parameters:

`T - the desired type of the wrapper, a single-method interface`

Parameters:

`intfc - a class object representing T`

`target - the method handle to invoke from the wrapper`

Returns:

`a correctly-typed wrapper for the given target`

Throws:

`NullPointerException - if either argument is null`

`IllegalArgumentException - if the intfc is not a valid argument to this method`

`WrongMethodTypeException - if the target cannot be converted to the type required by the requested interface`

isWrapperInstance

```
public static boolean isWrapperInstance(Object x)
```

Determines if the given object was produced by a call to `asInterfaceInstance`.

Parameters:

`x - any reference`

Returns:

`true if the reference is not null and points to an object produced by asInterfaceInstance`

wrapperInstanceTarget

```
public static MethodHandle wrapperInstanceTarget(Object x)
```

Produces or recovers a target method handle which is behaviorally equivalent to the unique method of this wrapper instance. The object `x` must have been produced by a call to `asInterfaceInstance`. This requirement may be tested via `isWrapperInstance`.

Parameters:

`x - any reference`

Returns:

`a method handle implementing the unique method`

Throws:

`IllegalArgumentException - if the reference x is not to a wrapper instance`

wrapperInstanceType

```
public static Class<?> wrapperInstanceType(Object x)
```

Recovers the unique single-method interface type for which this wrapper instance was created. The object `x` must have been produced by a call to `asInterfaceInstance`. This requirement may be tested via `isWrapperInstance`.

Parameters:

`x - any reference`

Returns:

`the single-method interface type for which the wrapper was created`

Throws:

`IllegalArgumentException` - if the reference x is not to a wrapper instance

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.